

Cahier des Charges

OptiEDT

AI-assisted generation, ranking and explanation of university timetables

Author	Mohamed Jawad Tourir		
Academic year	2025 – 2026	Duration	4 weeks
Project	Company internship project	Date	24 July 2026
Field of application	Tunisian public universities (LMD system)		

Contents

1. Introduction.....	4
1.1 Project context.....	4
1.2 Objectives	4
1.3 Project scope	4
2. General description of the project.....	5
2.1 Presentation	5
2.2 Study of the existing situation.....	5
2.3 Description of the actors	6
3. Scope of the project.....	6
3.1 Features included.....	6
3.2 Features outside the scope.....	7
3.3 Order of delivery	7
4. Functional specifications	8
4.1 Main modules.....	8
4.2 Functional requirements.....	9
4.3 User flow	10
4.4 AI recommendation and regeneration loop.....	10
4.5 AI assistant service	11
4.5.1 Why scoring and ranking stay outside the model	12
4.5.2 Grounding and verification.....	12
5. Constraint model.....	13
5.1 Hard constraints	13
5.2 Institutional calendar rules.....	13
5.3 Soft constraints	14
5.4 Weight profiles	14
5.5 Constraints of the examination session	15
6. Non-functional requirements.....	15
7. Technical architecture	17

7.1 Software architecture	17
7.2 Choice of the resolution method	18
7.3 Technologies used.....	18
7.4 Distribution of the intelligence in the system	19
8. Data strategy.....	20
8.1 Evaluation of the public sources	20
8.2 The generated instance.....	21
8.3 Verification of the instance	22
8.4 Data required by the adjustment of the weights.....	23
9. Constraints and assumptions	23
9.1 Technical constraints.....	23
9.2 Assumptions	23
10. Planning and phases of the project.....	23
11. Acceptance criteria	24
12. Risks and solutions.....	25
13. Appendices	25

List of figures

Figure 1. Actors and main use cases of the application.	5
Figure 2. Sequence from the data to the published timetable.	10
Figure 3. AI recommendation and regeneration loop.	11
Figure 4. AI assistant service, grounding and validation gates	12
Figure 5. System architecture and deployment.	17
Figure 6. Division of the responsibilities between the decision layer and the analysis layer.	18
Figure 7. Role given to each source and origin of the data used by the application.	22
Figure 8. Project workflow, phases of the first increment.	24

List of tables

Table 1. Description of the actors	6
Table 2. Order of delivery in two increments	7
Table 3. Functional requirements.....	9
Table 4. Hard constraints of the weekly model.....	13
Table 5. Soft constraints and default weights	14
Table 6. Constraints of the examination session	15
Table 7. Comparison of the methods for ranking and explanation.....	19
Table 8. Distribution of the techniques across the responsibilities	20
Table 9. Role retained for each data source	20
Table 10. Content of the generated instance.....	21
Table 11. Result of the five verifications of the instance.....	22
Table 12. Milestones of the project	24
Table 13. Risks and solutions	25

1. Introduction

1.1 Project context

At the beginning of each semester, every department of a Tunisian public university must build the timetable of its programmes. This work is done manually, most often in a spreadsheet, and requires checking after each placement that no teacher, no group and no room is used twice at the same moment. The operation takes two to three weeks, depends entirely on one person, and errors are frequently discovered after publication.

The difficulty does not stop when a timetable has been obtained. The person in charge holds one arrangement and has no means of knowing whether a better one exists, nor any argument to give to a teacher who contests a placement. The same work is then done a second time for the examination session. The proposed application automates the construction, produces several arrangements instead of one, states in verifiable terms why one of them is preferred, and covers the examination session with the same engine.

1.2 Objectives

- **Study the available data** : evaluate the public datasets before implementation, state what can be reused, what cannot, and why.
- **Automate the generation** : produce a valid weekly timetable without manual verification of conflicts.
- **Guarantee the absence of conflicts** : no teacher, group or room used twice in the same slot.
- **Compare several solutions** : produce several candidate timetables, each with a score out of 100, ordered by that score.
- **Explain and recommend** : state the contribution of each quality criterion to the difference between two candidates, and recommend one of them.
- **Optimise iteratively** : let the person in charge accept a recommendation and trigger an automatic regeneration of the timetable, without ever relaxing a hard constraint.
- **Assist in ordinary language** : integrate a language model service which explains the quality of a timetable, answers questions about a run and produces readable reports, working only from figures already computed.
- **Assist in ordinary language** : an AI service explains the quality of a timetable, answers questions about it and produces a readable report, working only from figures the application has already computed.
- **Collect teacher availability** : replace exchanges by email with a web form.
- **Cover the academic period** : the weekly timetable of the semester and the timetable of the examination session.
- **Adapt to the institution** : holidays, closed slots, Ramadan window and LMD structure held as configuration data and not written into the code.

1.3 Project scope

The application will allow :

- Teachers to declare their availability and consult their personal timetable.

- The person in charge of the timetable to generate several candidates, compare them, read the reason for their order and publish the chosen one.
- Students to consult and print the timetable of their group and of their examinations.
- The administrator to manage the accounts, the holidays and the academic calendar.
- Any authorised user to consult the occupancy of a classroom or of a laboratory.

2. General description of the project

2.1 Presentation

The application is a web platform which connects four types of users :

- **Person in charge of the timetable** : loads the data, launches the generation, compares the candidates and publishes the result.
- **Teachers** : declare their availability and consult their schedule.
- **Students** : consult the timetable of their group.
- **Administrator** : manages the accounts, the holidays and the calendar.

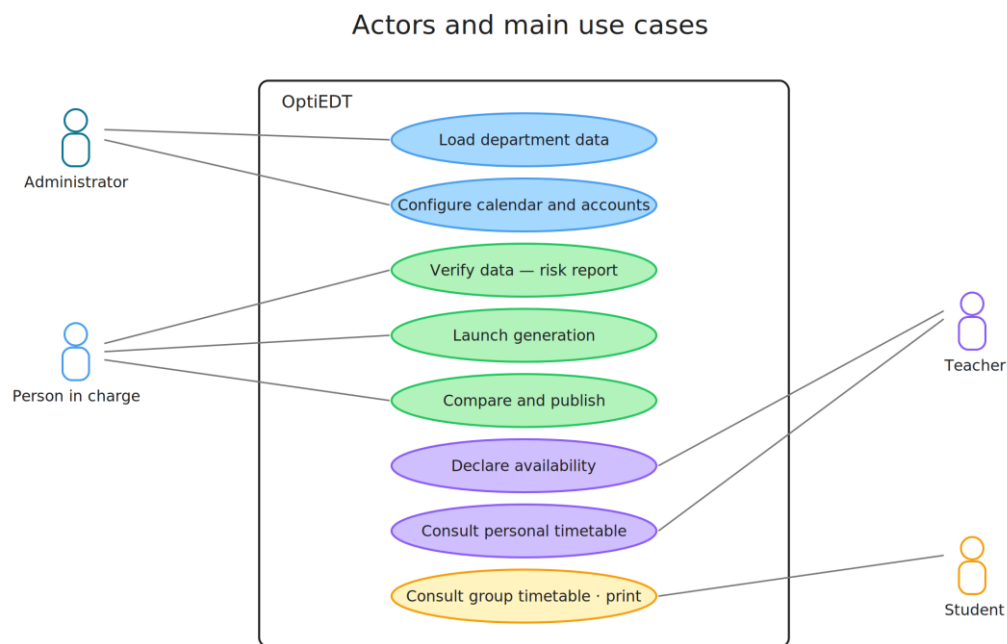


Figure 1. Actors and main use cases of the application.

2.2 Study of the existing situation

Limits of manual scheduling :

- Verification of conflicts done by eye, unreliable when the number of sessions increases.
- Teacher preferences collected by email and frequently lost.
- A single timetable produced, without comparison and without optimisation.

- No trace of the reasons which led to a placement, so that a contested choice cannot be defended.
- The examination session is built separately, by the same manual method and with the same risks.
- Two to three weeks of work for each semester.

Limits of the existing software (UniTime, Scientia) :

- No native representation of the CM, TD and TP hierarchy of the LMD system.
- No management of a lunar calendar nor of a period during which the working day is shortened.
- No management of the closed slots specific to a Tunisian institution.
- A solution is returned, but the compromise which it represents is not exposed in a form which the department can discuss.
- Cost of adaptation higher than the development of a dedicated application.

2.3 Description of the actors

Table 1. Description of the actors

Actor	Description
Person in charge of the timetable	Loads the data, launches the generation, compares the candidates and publishes the chosen timetable.
Teacher	Declares availability and consults the personal timetable.
Student	Consults and prints the timetable of the group.
Head of department	Examines the teaching loads and approves the timetable.
Administrator	Manages the accounts, the holidays and the academic calendar.

3. Scope of the project

3.1 Features included

For the person in charge of the timetable :

- Loading of the data of the department : courses, groups, teachers, rooms and slots.
- Verification of the data before solving and reading of the report of structural risks.
- Launch of the generation with a configurable time limit.
- Comparison of the candidates by score, criterion by criterion, and reading of the contribution of each criterion to the difference.
- Publication of the chosen timetable.
- Consultation of the report naming the rules in conflict when no timetable exists.
- Reading of the AI recommendations attached to a candidate, and, on acceptance, automatic regeneration of an improved candidate.
- Consultation of the AI assistant : explanation of the quality of a candidate, answers to questions asked in ordinary language, and generation of a readable report on a run.

For the teachers :

- Declaration of availability on a weekly grid.
- Consultation of the personal timetable and of the examinations supervised.

For the students :

- Consultation of the timetable of the group and of the examination session.
- Printing and export of the displayed view.

General features :

- Authentication and management of access rights by role.
- Configuration of the calendar : holidays, closed slots and the period during which the working day is shortened.
- Display of the timetable by teacher, by group, by classroom and by laboratory.
- Record of every generation run, with its data, its parameters and its results.

3.2 Features outside the scope

- Manual modification of the published timetable by drag and drop.
- Re-solving after a modification of availability or of a room once the timetable is published.
- Entry of constraints in ordinary language. The published approaches which translate a sentence into a constraint model obtain acceptable results only when the generated model is validated by a solver in a loop; building that validation harness does not fit the duration of the project, and a sentence understood incorrectly produces a timetable which is wrong without being detectably wrong.
- Generation of the timetable by a learned model, for the reasons stated in section 7.2.
- Interface in Arabic.
- Statistics on the use of the rooms.
- Simultaneous treatment of several faculties of the same university.
- Mobile application.

3.3 Order of delivery

The features are delivered in two increments. The first is the object of the commitment taken in this document. The second is specified and designed together with the first, so that it can be undertaken without a new study.

Table 2. Order of delivery in two increments

Increment	Content	Status
Increment 1	Data, availability, verification before solving, generation of several candidates, score, ranking, explanation of the difference, AI assistant service,	Committed

Increment	Content	Status
	recommendations with accept-and-regenerate, views, publication, rights, record of the runs	
Increment 2	Examination session and adjustment of the weights from the comparisons made	Conditional on the duration

4. Functional specifications

4.1 Main modules

- **Authentication module** : management of the accounts and of the access rights by role.
- **Data management module** : courses, groups, teachers, rooms, slots and holidays, loaded from forms or from files with a report of the rejected lines.
- **Calendar module** : closed slots, holidays and the shortened-day period, held as configuration and applied by closing slots or by shifting the hours displayed.
- **Availability module** : weekly grid filled in by the teachers.
- **Pre-analysis module** : verifications applied to the data before any solving, and report of the structural risks found.
- **Solving module** : construction of the timetables by the CP-SAT engine, executed once per weight profile.
- **Diagnosis module** : second solving run, entered only when the first concludes that no timetable exists, which names the rules in conflict.
- **Scoring module** : calculation of the sub-scores, of the score out of 100 and of the order of the candidates.
- **Explanation module** : decomposition of the difference between two candidates and formulation of the recommendation.
- **Regeneration module** : translates an accepted recommendation into a solver input, a weight adjustment, a locked session or an excluded slot, and launches a new run through the same engine.
- **AI assistant service** : hosted language model reached through a provider-agnostic client, which explains the quality of a candidate, answers questions on it, phrases the recommendations and drafts a report, from the figures computed by the scoring and explanation modules.
- **Examination module** : placement of the examinations of a session (increment 2).
- **AI assistant module** : calls an external language model to explain a candidate, answer a question on a run and produce a report, from a context of figures already computed.
- **Learning module** : adjustment of the weights of the criteria from the comparisons made by the person in charge (increment 2).
- **Display module** : views by teacher, by group, by classroom, by laboratory and by examination, printing and export.
- **Audit module** : record of every generation run and of the parameters which produced it.

4.2 Functional requirements

Table 3. Functional requirements

Code	Requirement	Priority
FR-1	Load and manage the data of the department	Necessary
FR-2	Allow a teacher to declare availability on a weekly grid	Necessary
FR-3	Generate a timetable respecting the hard constraints H1 to H12	Necessary
FR-4	Improve the quality criteria within a time limit	Necessary
FR-5	Produce several candidates and give each a score out of 100	Necessary
FR-6	Order the candidates by score	Necessary
FR-7	Display the timetable by teacher, by group and by room	Necessary
FR-8	Report the rules in conflict when no timetable exists, by the verifications of the data and by the explanation of infeasibility returned by the solver	Necessary
FR-9	Configure the calendar : holidays, closed slots and shortened-day period	Necessary
FR-10	Print or export a timetable view	Expected
FR-11	Authenticate the users and restrict the access by role	Necessary
FR-12	Verify the data before solving and report the structural risks found	Necessary
FR-13	Produce the candidates under distinct weight profiles	Necessary
FR-14	Compare two candidates criterion by criterion on the same screen	Necessary
FR-15	State the contribution of each criterion to the difference between two candidates	Necessary
FR-16	Recommend one candidate and state the rule which produced the recommendation	Expected
FR-17	Signal a recommended candidate which another candidate improves on every criterion	Expected
FR-18	Display the occupancy of each classroom and of each laboratory	Expected
FR-19	Record every generation run with its data, its seed, its weights and its results	Necessary
FR-20	Generate the timetable of an examination session	Expected (increment 2)
FR-21	Adjust the weights of the criteria from the comparisons made	Optional (increment 2)
FR-22	Explain in ordinary language the quality of a candidate, from figures already computed	Necessary
FR-23	Regenerate a timetable from an accepted recommendation, preserving every hard constraint	Necessary
FR-24	Answer a question asked in ordinary language about a run or a candidate	Necessary
FR-25	Produce a readable report on a run, ready to be printed or circulated	Expected

Code	Requirement	Priority
FR-24	Answer a question asked in ordinary language about a published or candidate timetable	Necessary
FR-25	Produce a readable report on a run and on the candidate retained	Expected

4.3 User flow

1. The administrator loads the data of the department and configures the calendar.
2. The teachers declare their availability on the weekly grid.
3. The application verifies the data and reports the structural risks before any solving.
4. The person in charge launches the generation with a time limit.
5. The engine produces several candidate timetables under distinct weight profiles.
6. Each candidate receives sub-scores and a score out of 100, and the candidates are ordered.
7. The person in charge compares two candidates, reads the contribution of each criterion to the difference, and publishes the chosen one.
8. The teachers and the students consult and print their timetable.

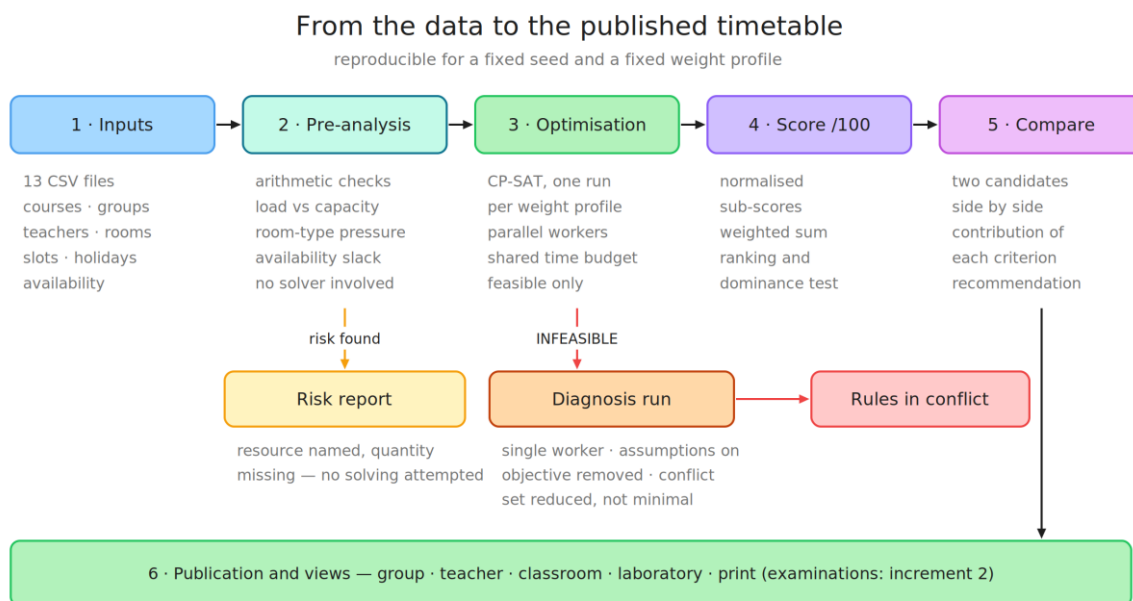


Figure 2. Sequence from the data to the published timetable.

4.4 AI recommendation and regeneration loop

The AI module does not build timetables. It reads the candidates already produced by the engine and behaves as an analyst : it scores, ranks, compares, explains and, where it finds an opportunity, proposes a recommendation.

A recommendation is always one of a closed list of actions, never a free instruction :

- **Weight adjustment (weight_delta)** : raise or lower the weight of one soft criterion for the next run.

- **Lock a session (lock_session)** : fix a session to its current slot and room while the rest is re-optimised.
- **Exclude a slot or a room (exclude_slot)** : remove one option for one session before the next run.

These three names are the identifiers used throughout the specification, in particular in the recommendation catalogue of the Software Requirements Specification, section 6.10.

If the person in charge accepts a recommendation, the regeneration module translates it into that solver input and launches a new run of the same CP-SAT engine used for the first generation, as shown in *Figure 3*. Nothing is edited by hand and nothing is edited by the AI : the new candidate is produced by the same construction as the first one, so it satisfies H1 to H12 for the same reason the first candidate did. If the recommendation is refused, the current candidate is kept unchanged.

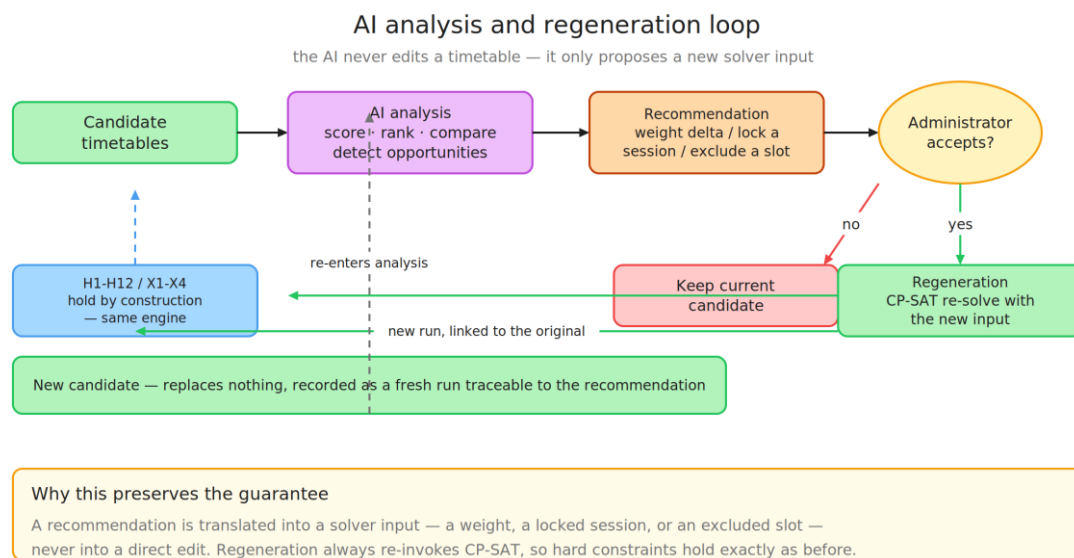


Figure 3. AI recommendation and regeneration loop.

4.5 AI assistant service

The application integrates a hosted language model, reached through a provider-agnostic client. It is a committed part of the first increment, and its role is bounded by one rule : it works on figures the application has already computed, and it produces text.

What the service does :

- Explains in ordinary language why a candidate scores as it does, and where its weaknesses are.
- Answers a question asked in ordinary language about a candidate or a published timetable.
- Phrases the recommendations produced by the analysis layer.
- Drafts a readable report on a run and on the candidate retained.

What the service does not do :

- It does not place a session, and it never writes into a timetable.

- It does not compute a score and it does not decide a ranking. Both are computed exactly, and the published evaluations of model-based scoring report position, verbosity and self-preference biases, with verdicts that flip when two candidates are simply swapped in the prompt. A figure obtained that way could not be defended before a department.
- It does not receive the database. It receives a payload built by the application.

4.5.1 Why scoring and ranking stay outside the model

The internship notes ask the AI to score and to rank. Both are arithmetic : the department must be able to recompute a score by hand, and the same data must give the same order twice. A language model offers neither property. The platform therefore computes the score and the order by the weighted sum of section 5.6, and the model explains the result, answers questions about it and writes it up. The requirement is met in substance and the figures stay verifiable.

4.5.2 Grounding and verification

9. The context builder selects, from the run, only the figures needed to answer : sub-scores, contributions, placements concerned and weights in force.
10. That context is sent to the model together with the question, and nothing else is sent.
11. The answer is checked : it must respect the expected structure, every figure it states must appear in the context supplied, and every identifier it names must exist in the instance.
12. An answer which fails a check is not displayed; the computed form is shown instead, as when the service is unreachable.

The check detects a figure the model invented. It does not establish that the rest of the sentence is correct, and this document claims no more than that. That limit is precisely why no decision the department has to defend depends on this service. The service and its checks are shown in *Figure 4*.

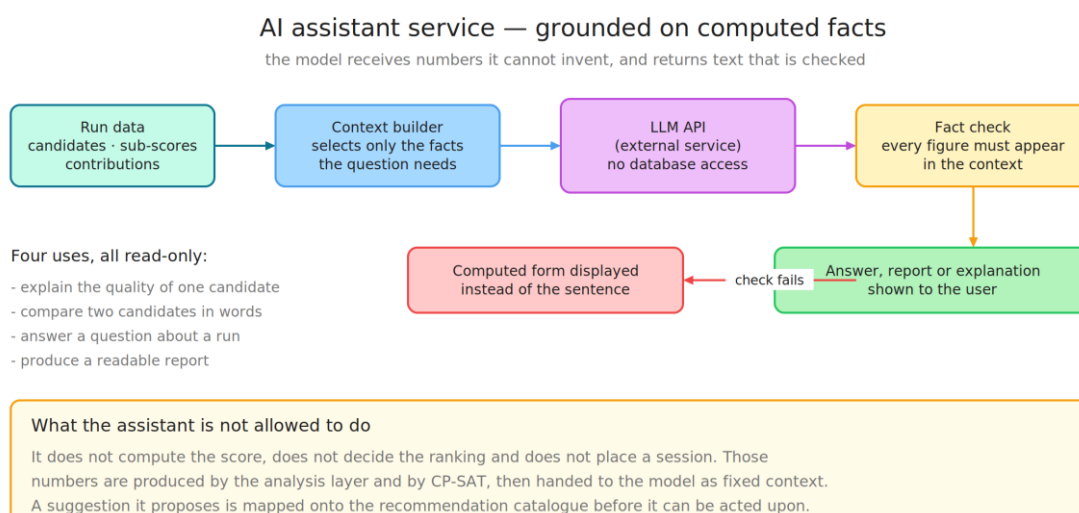


Figure 4. AI assistant service, grounding and validation gates

5. Constraint model

5.1 Hard constraints

These constraints must be respected in every accepted timetable. They are held in the file `constraint_catalogue.csv` of the instance, where each one carries a reference to the corresponding constraint type of the XHSTT archive when such a type exists.

Table 4. Hard constraints of the weekly model

Code	Statement	XHSTT reference
H1	A teacher has at most one session per slot	AvoidClashesConstraint
H2	A group has at most one session per slot	AvoidClashesConstraint
H3	A room hosts at most one session per slot	AvoidClashesConstraint
H4	The session is placed in a room of the required type	PreferResourcesConstraint
H5	The capacity of the room covers the size of the group	—
H6	The unavailability declared by a teacher is respected	AvoidUnavailableTimesConstraint
H7	Each session receives exactly one slot and one room	AssignTimeConstraint
H8	A session of several periods stays within one day	—
H9	No session is placed on a closed slot or on a holiday	AvoidUnavailableTimesConstraint
H10	A session marked as locked keeps the slot given to it	AssignTimeConstraint
H11	The aggregate demand does not exceed the capacity of the rooms	—
H12	A promotion and its groups and subgroups are never busy at the same time	LinkEventsConstraint

Note on H12 : in the LMD organisation a lecture gathers the whole promotion, while the tutorials and the laboratory sessions concern its groups and subgroups. When a lecture is placed, the students of every subgroup are occupied, and no session of these subgroups can be placed in the same slot. This constraint is the equivalent, for a hierarchy of groups, of the conflict constraint of the curriculum-based formulation of the international competition.

5.2 Institutional calendar rules

The shortening of the working day during Ramadan and the closing of certain half-days are rules of the institution and not properties of the timetabling problem. They are therefore not written as constraints of the model. They are held as configuration in the file `calendar_config.csv` and they act in two ways only :

- **By closing slots :** a closed half-day sets the field `is_open` of the slots concerned to zero, and the constraint H9 then removes those slots from the domain of the sessions. No new rule is needed.

- **By shifting the hours** : during the declared period the start and end hours of the periods are shifted by the configured number of minutes. The index of the slot does not change, so the model is unaffected and only the hours displayed and printed differ.

The configuration currently supplied with the instance declares six days of five periods, the two afternoon periods of Saturday closed, the Friday afternoon open, a Ramadan window from 18 February to 19 March 2026 and a shift of sixty minutes. Twenty-eight slots out of thirty are therefore open. Changing any of these values changes the timetable produced without any modification of the code.

If the institution confirms that a rule of this kind must never be violated under any circumstance, it is promoted to a hard constraint of the model and receives its own code in the catalogue. Until that confirmation is obtained, treating it as configuration is the safer choice : a rule written into the model cannot be relaxed by the user, whereas a rule held as data can be adjusted by the department itself.

5.3 Soft constraints

These constraints are not imposed. Each violation adds a penalty to the cost which the engine tries to reduce. The weights below are the default values of the catalogue supplied with the instance.

Table 5. Soft constraints and default weights

Code	Statement	Default weight
S2	Minimise the idle time inside the day of a group	0.25
S3	Minimise the idle time inside the day of a teacher	0.15
S4	Reduce the number of distinct days of presence	0.10
S5	Honour the preferred windows declared by a teacher	0.20
S6	Avoid rooms which are markedly under-used or over-used	0.10
S7	Spread the sessions of one course across the week	0.10
S10	Keep a midday break where possible	0.00

The default weights sum to 0.90. The scoring module normalises the weights of a profile so that their sum equals one before any score is computed, which makes the scores of two runs comparable. A criterion whose weight is zero, such as S10, is measured and displayed but does not influence the order.

5.4 Weight profiles

A weight profile is a vector of non-negative weights over the soft constraints whose sum equals one. The same data solved under two different profiles gives two different timetables, both valid. The application uses at least three profiles, so that the candidates presented differ in a way which can be named : a balanced profile, a profile which favours the students by raising the weight of S2, and a profile which favours the teachers by raising the weights of S3 and S5.

Producing the candidates under distinct profiles rather than under distinct random seeds alone is a deliberate choice. Two candidates obtained by changing only the seed differ without a reason which can be stated, whereas two candidates obtained under two profiles differ for a reason which is exactly the difference between the two profiles.

5.5 Constraints of the examination session

The examination session reuses the same engine with its own constraints. This part belongs to the second increment and is stated here so that the model is complete.

Table 6. Constraints of the examination session

Code	Statement
X1	A student never has two examinations in the same slot
X2	The capacity of the rooms assigned to an examination covers the number of candidates
X3	An examination is placed inside the period of the session and outside the holidays
X4	A supervisor is not assigned to two examinations in the same slot
SX1	The examinations of the same group are spread over the session (soft)

The examinations are the only part of the system where the individual student matters. For the weekly timetable a promotion follows a fixed programme, so reasoning on groups loses no information; for an examination session two students of the same group may present different optional courses, so the conflicts are computed from the file `students.csv`, which holds 425 students each attached to a promotion, a tutorial group and a laboratory subgroup.

6. Non-functional requirements

- **Reliability** : if a timetable exists the application returns one, otherwise it names the rules responsible.
- **Guarantee under regeneration** : a candidate produced from an accepted recommendation is obtained by a fresh call to the same solver, so it satisfies H1 to H12 for the same reason as the first candidate, never by a direct edit.
- **Grounding of the assistant** : every figure produced by the language model is checked against the context supplied to it. A figure absent from that context is not displayed.
- **Independence from the assistant** : if the language service is unreachable, generation, scoring, ranking, comparison and publication all continue to work, and only the text disappears.
- **Reproducibility** : for the same data, the same weights and the same random seed, two executions give the same result. The seed, the weights and the version of the model are recorded with each run.
- **Explicability** : every score displayed can be recomputed by hand from the sub-scores and the weights recorded with the run.
- **Separation of the responsibilities** : no learned component and no language model takes part in the assignment of a session to a slot or to a room, nor in the computation of a score or of a ranking.

- **Grounding of the AI service** : the language model receives only the figures computed by the application, never a direct access to the database. Its output is checked before display : any figure it states must appear in the payload it received, and any identifier it names must exist.
- **Data minimisation** : the payload sent to the AI service carries aggregated figures, codes and labels. No student name, no teacher email address and no personal identifier leaves the institution.
- **Human control** : no timetable becomes visible to the teachers and to the students without an explicit action of publication by the person in charge.
- **Performance** : the targets are a first valid timetable in less than 60 seconds and a portfolio of three candidates within a limit of 5 minutes on the reference instance. These figures are objectives to be measured during the second phase and not guarantees, the problem being of a class for which no bound on the solving time can be announced in advance.
- **Degraded mode** : if the AI service is unreachable, refuses or fails a check, the application falls back to the computed form of the explanation. Generation, scoring, ranking, comparison and publication remain available in full.
- **Latency of the assistant** : an explanation or an answer is returned in less than 10 seconds, the request being asynchronous so that the interface is never blocked.
- **Capacity** : one faculty, that is about 220 sessions, 45 teachers and 20 rooms.
- **Security** : authentication by token, hashed passwords, verification of rights on the server side.
- **Protection of personal data** : a teacher is identified by an account, a student by the group to which the student belongs. No mark, no individual result and no information foreign to the construction of the timetable is recorded.
- **Maintainability** : calendar, slots and constraint catalogue held as data, so that the application can be reused the following year.
- **Usability** : availability grid understandable without training and filled in in less than 5 minutes.

7. Technical architecture

7.1 Software architecture

The application is organised in four layers, deployed as a single unit and detailed in the Software Requirements Specification, section 2.1.

- **Presentation layer** : web interface, accessible from a browser without installation on the user side, displaying the grids, the comparison of the candidates and the timetable views.
- **Application layer** : server exposing a REST interface for the data, the generation and the comparison, and responsible for the relational database of courses, groups, teachers, rooms, slots, timetables and generation runs.
- **Decision layer** : the CP-SAT solving engine, called by the server in the same execution environment but executed as a background task, since a generation exceeds the duration a request can hold open.
- **Analysis layer** : components which read the results of the engine and produce the scores, the order and the explanations, without any access to the solving model.

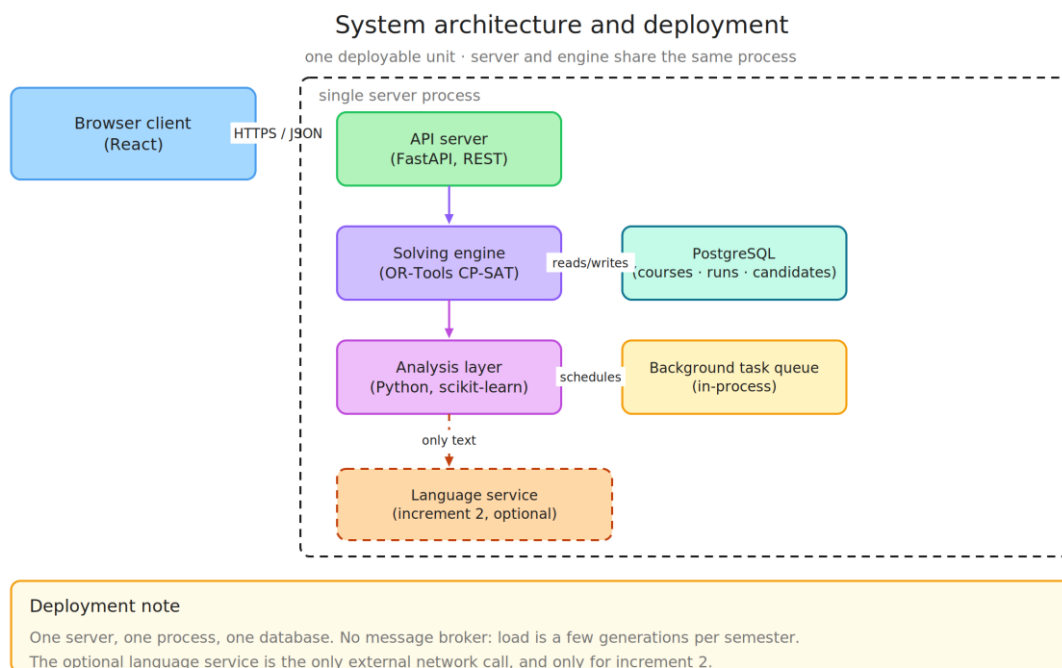


Figure 5. System architecture and deployment.

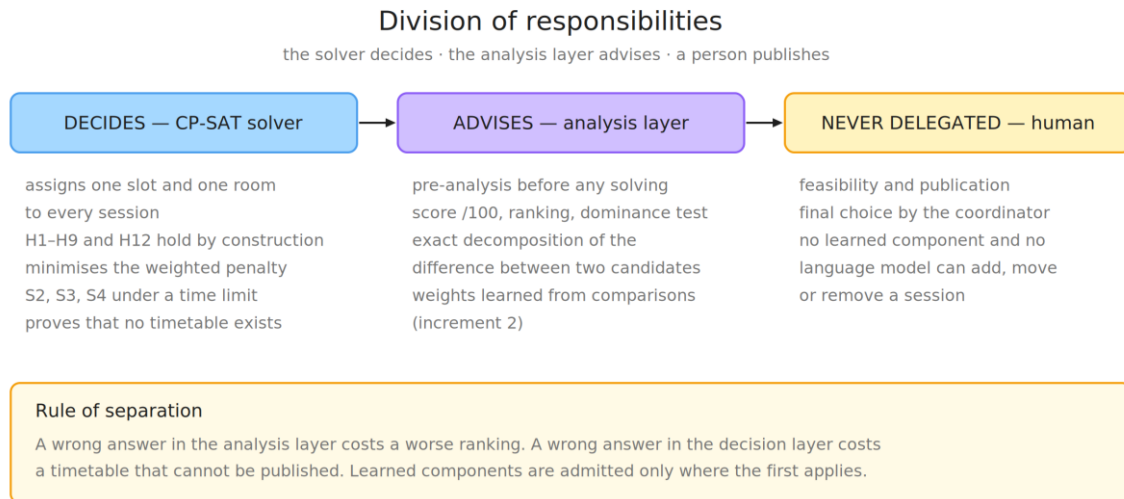


Figure 6. Division of the responsibilities between the decision layer and the analysis layer.

7.2 Choice of the resolution method

Four families of methods were compared before choosing the method which assigns a slot and a room to each session : metaheuristics, integer linear programming, machine learning and constraint programming. The comparison, conducted against four criteria (guarantee of feasibility, detection of infeasibility, need for training data, effort of implementation), is detailed in the Project Plan and Methodology, section 4.4.

Constraint programming was retained : it is the only one of the four which guarantees, by construction, that no timetable containing a conflict can be produced, and it is able to establish that no solution exists rather than searching indefinitely. The CP-SAT solver of the OR-Tools library implements it. It is free, maintained, offers a Python interface, and accepts a time limit by returning the best solution found.

7.3 Technologies used

- **Solving engine** : OR-Tools CP-SAT, chosen for the guarantee of feasibility and its Python interface.
- **Back-end** : FastAPI in Python, so as to call the engine directly and benefit from automatic data validation.
- **Database** : PostgreSQL, for the consistency of the links between sessions, groups, teachers and rooms.
- **Analysis components** : the standard scientific libraries of Python for the scores and the decomposition, and scikit-learn for the adjustment of the weights in the second increment.
- **Front-end** : React, whose component model corresponds to the display of weekly grids and of comparison views.
- **Deployment** : a single application, the server and the engine sharing the same environment, the generation being executed as a background task.

7.4 Distribution of the intelligence in the system

The company expects a platform assisted by artificial intelligence and not only a solver. The requirement is satisfied by placing each technique where it is defensible, and not by renaming the existing engine.

The system contains two decisions of a different nature. The first is the assignment of the sessions, which admits no error : a timetable containing a conflict cannot be published, whatever its other qualities. The second is the choice among timetables which are all valid, which is a question of compromise and not of correctness : an error there produces a worse ranking, not an unusable result. The methods which offer a guarantee are placed on the first decision, the methods which learn are placed on the second.

Constraint programming, retained for the first decision, is itself a method of artificial intelligence in the accepted sense of the term : it belongs to the symbolic branch of the discipline, that of constraint satisfaction problems, and it is presented as such in the reference literature. The application therefore uses artificial intelligence at both levels, of two different kinds, and this document names them separately rather than using one word for both.

A second point deserves to be stated, because the two objectives of the project appear to contradict each other. Producing the best timetable and ranking several timetables are not the same demand. The solver proves optimality with respect to a given weight vector; the weight vector expresses what the department considers a good timetable and no faculty has ever written it down. The application therefore produces the proven optimum under several plausible weightings and asks a person to choose. Ranking exists because the objective is uncertain, not because the engine is weak.

Four methods were compared for the second decision, that of ranking the valid candidates and of justifying the order obtained.

Table 7. Comparison of the methods for ranking and explanation

Method	Reproducible	Data required	Explains its result	Decision
Weights fixed with the department	Yes	None	Exactly	Retained as the basis
Weights learned from the comparisons made	Yes	Produced by the application	The weights are visible	Retained for increment 2
Ranking by a neural model	Partly	A large set of examples	No	Rejected
Ranking by a language model	No	None	No	Rejected for the ranking, retained for the explanation

The weighted sum was retained because the score is a linear function of the criteria. This is not a detail of presentation : it means that the difference between the scores of two candidates decomposes exactly into the sum of the contributions of each criterion, so the explanation given to the user is the calculation itself and not an approximation of it. A ranking produced by a neural

model or by a language model would require an explanation constructed after the fact, which the department could not verify.

The distribution finally retained is given in *Table 8*.

Table 8. Distribution of the techniques across the responsibilities

Responsibility	Technique used
Assignment of the sessions to the slots and to the rooms	Constraint programming, CP-SAT
Respect of the hard constraints	Constraint programming, guaranteed by construction
Reduction of the penalties of the soft constraints	Optimisation under a time limit, CP-SAT
Detection of the impossible situations	Arithmetic verification of the data before solving
Naming of the rules in conflict	Explanation of infeasibility returned by the solver in a separate run
Diversity of the candidates presented	Repeated solving under distinct weight profiles
Score, order and decomposition of the difference	Weighted sum of normalised criteria, exact calculation
Adjustment of the weights to the practice of the department	Supervised learning on the comparisons made by the person in charge
Explanation in ordinary language, questions and reports	Hosted language model applied to figures already computed, under validation

The adjustment of the weights belongs to the second increment. The AI service belongs to the first : it receives a structured payload of computed figures and returns text. It does not read the database, does not modify a timetable, does not compute a score and takes no part in any ranking. If it is unavailable, the explanation remains readable in its computed form.

8. Data strategy

8.1 Evaluation of the public sources

Three public sources were evaluated before any implementation : the curriculum-based track of the ITC-2007 competition, the XHSTT-2014 constraint archive, and a Kaggle examination-scheduling dataset. None could be merged with another — their identifiers do not correspond, their level of detail differs, and two files of the Kaggle dataset are invalid. The full evaluation, including the verification performed on each source and the reason no merge was attempted, is in the Project Plan and Methodology, section 3.

Table 9. Role retained for each data source

Source	Role retained	Verified
ITC-2007, Track 3	Validation of the course engine against published results	Yes
XHSTT-2014	Catalogue of the types of constraints to be supported	Yes

Source	Role retained	Verified
Kaggle, University Exam Scheduling	Reference for the model of the examination session	Yes, partly usable
ITC-2007, Track 1	Validation of the examination engine	Not yet, increment 2
Generated instance	Data on which the application runs and is demonstrated	Yes

The reading of the XHSTT archive produced a concrete deliverable rather than an import : the file `constraint_catalogue.csv`, which lists nineteen constraints, twelve hard and seven soft, each with the name of the corresponding XHSTT constraint type when one exists. That file is the link between the published formalism and the model of this project.

8.2 The generated instance

No public dataset contains the CM, TD and TP structure, the hierarchy between a promotion and its subgroups, the Tunisian calendar or the availability of the teachers. A generator was therefore written for this project. The instance produced is described below and is the only data loaded into the database.

Table 10. Content of the generated instance

Element	Content of the instance
Programmes	3 : Licence Informatique, Licence Mathématiques, Master Informatique
Promotions	6, from L1 to M1, from 40 to 120 students
Groups	51 : 6 promotions, 15 tutorial groups, 30 laboratory subgroups
Students	425, each attached to a promotion, a tutorial group and a subgroup
Courses	32, all of the second semester
Sessions	218 : 32 lectures, 82 tutorials, 104 laboratory sessions
Duration of the sessions	114 of one period, 104 of two consecutive periods
Teachers	44 : 7 professors, 11 senior lecturers, 13 assistant lecturers, 13 assistants
Maximum loads	9, 12, 18 or 24 hours per week according to the rank
Rooms	20 : 2 lecture theatres, 10 classrooms, 6 computer laboratories, 2 science laboratories
Capacities of the rooms	from 20 to 250 places
Slot grid	6 days of 5 periods of 90 minutes, that is 30 slots, of which 28 are open
Closed slots	the two afternoon periods of Saturday
Holidays	18 days blocking scheduling, of which 10 follow the lunar calendar
Shortened-day period	18 February to 19 March 2026, shift of 60 minutes
Declared availability	157 declarations of unavailability covering 41 of the 44 teachers

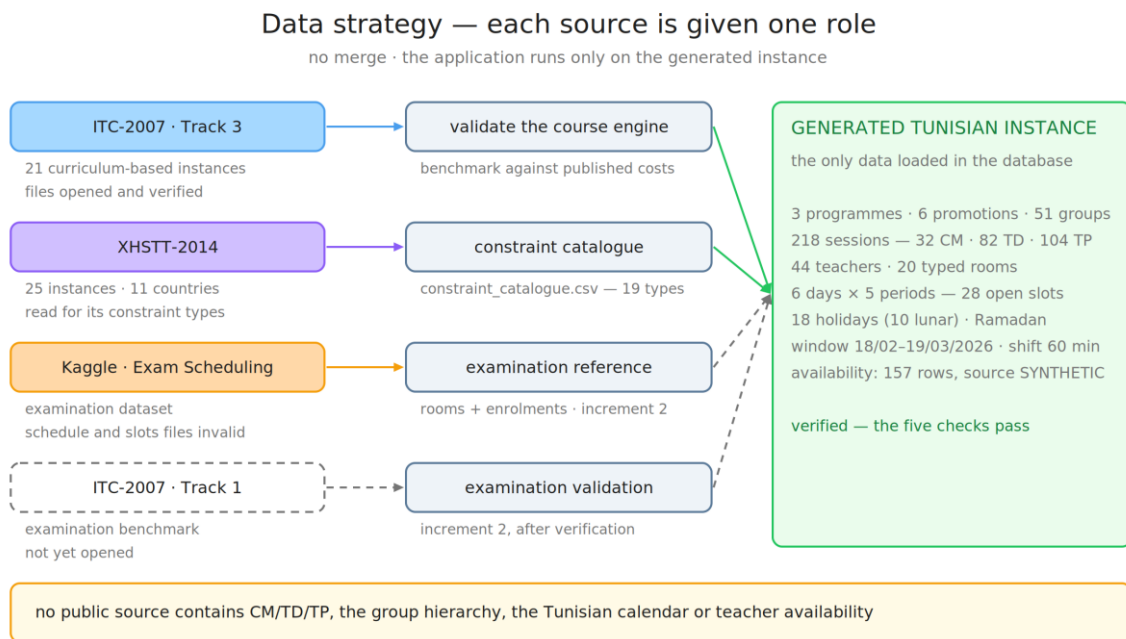


Figure 7. Role given to each source and origin of the data used by the application.

8.3 Verification of the instance

The instance is verified before any solving by five checks, whose purpose is to distinguish a problem which genuinely has no solution from an error in the model. All five pass on the instance supplied.

Table 11. Result of the five verifications of the instance

Verification	Result obtained
Each session can find a room of the required type and of sufficient capacity	0 session without a suitable room
The open slots cover the demand for each type of room	lecture theatres 57%, classrooms 29%, computer laboratories 95%, science laboratories 86%
No teacher is assigned more hours than the maximum load of the rank	0 teacher above the limit, the heaviest load being 12 periods, that is 18 hours
Each teacher keeps enough free slots for the sessions assigned	0 teacher in difficulty, the smallest margin being 11 free slots
The hierarchy of the groups is consistent	0 invalid reference, 0 invalid chain, the 425 students matching the declared group sizes exactly

The occupancy of the computer laboratories, at 95 per cent of the available capacity, is the tightest point of the instance and the one most likely to make the problem infeasible if a room is withdrawn. This figure is recorded here because it is the number to watch when the instance is modified.

8.4 Data required by the adjustment of the weights

Adjusting the weights automatically would require a collection of timetables accepted or refused by a department, with the reason for the decision. No such collection exists for a Tunisian faculty, and it cannot be reconstructed after the fact, since the published timetables of past years carry no trace of the alternatives which were set aside.

The weights therefore start at the values held in the catalogue, and the comparisons made by the person in charge on the comparison screen constitute, run after run, the only set of examples on which an adjustment can later be based. The application is, from its first use, the instrument which produces the data that the second increment needs.

9. Constraints and assumptions

9.1 Technical constraints

- Execution on the standard equipment of an institution, without a graphics card dedicated to computation.
- Solving deterministic and reproducible for a fixed random seed and a fixed weight profile.
- Generation executed as a background task, the interface following the state of the run.
- Calendar, slots and constraint catalogue held as data and not written into the code.
- Access to the data restricted according to the role of the user.
- The AI service is reached through a provider-agnostic client and can be disabled by configuration without affecting any other function.
- No graphics card being available, the language model is consumed as a hosted service rather than self-hosted.

9.2 Assumptions

- The institution considered is a public faculty organised according to the LMD structure.
- The period treated is the second semester of the academic year 2025-2026.
- The composition of the groups is fixed for the whole semester.
- The list of rooms and the enrolments are supplied by the institution.
- The data used for the demonstration is generated, no real data being available. The availability file carries the mention SYNTHETIC in its source column so that generated declarations are never confused with declarations made by a teacher.
- No collection of validated timetables exists, so the weights of the criteria start at the values held in the catalogue.
- The examinations of a session concern the same population as the semester, and their supervision is assigned by the department.

10. Planning and phases of the project

The project is organised in six phases over the twenty working days of the first increment. The detailed breakdown of tasks and duration per phase is in the Project Plan and Methodology,

section 8.1; the table below states the condition that must be reached at the end of each phase for the project to stay on schedule.

Table 12. Milestones of the project

Milestone	Condition to be reached
End of phase 1 (day 3)	Instance verified, evaluation of the sources written, scope fixed
End of phase 2 (day 8)	A timetable without conflict obtained on the instance
End of phase 3 (day 11)	Several candidates produced, ordered, and the difference between two of them decomposed
End of phase 4 (day 15)	Complete path from the declaration of availability to the publication
End of phase 5 (day 17)	An infeasible instance produces a report naming the rules in conflict
End of phase 6 (day 20)	Application demonstrable and documents completed

The second increment is estimated separately and is conditional on the duration available : four days for the examination session, three days for the adjustment of the weights and their evaluation, two days for the formulation in ordinary language.

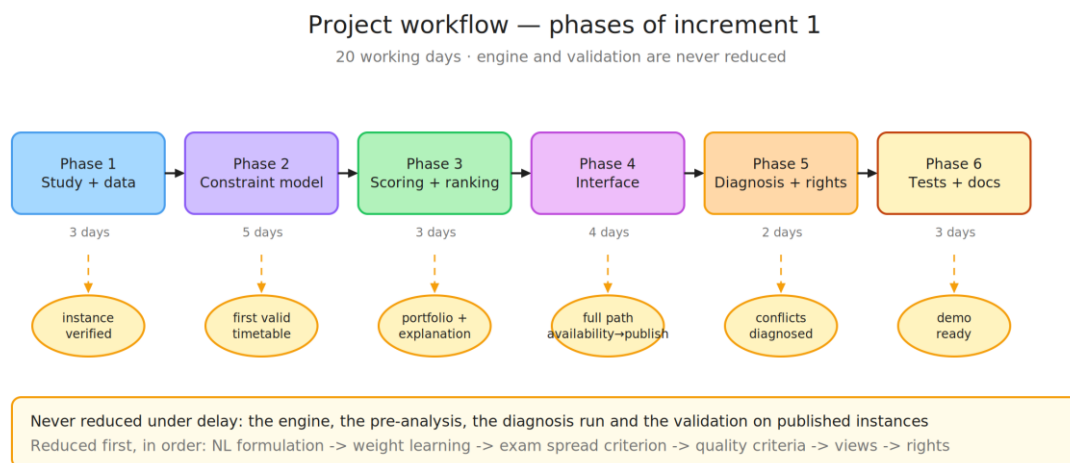


Figure 8. Project workflow, phases of the first increment.

11. Acceptance criteria

- The timetable produced on the reference instance contains no violation of a hard constraint.
- At least three candidates are produced, each displaying its overall score and the sub-scores which compose it.
- The sum of the contributions displayed for a comparison equals the difference between the two scores, to the precision of the display.
- Two executions with the same data, the same weights and the same seed produce the same candidates in the same order.
- An instance without solution produces a report naming the rules in conflict, obtained from the verifications of the data and from the explanation returned by the solver.

- A teacher account obtains only its own availability and its own timetable.
- Closing a half-day in the calendar configuration removes the corresponding slots from every timetable produced, without any modification of the code.
- The availability grid is filled in in less than 5 minutes without training.
- Every published timetable can be traced back to the run, the seed and the weights which produced it.

12. Risks and solutions

The engineering risks of the project — modelling delay, solving time, dataset saturation, and the risk of an unrealistic instance — are tracked in the Project Plan and Methodology, section 9, together with their effect and mitigation. The table below lists the risks specific to the scope and to the AI components of this document.

Table 13. Risks and solutions

Risk	Solution
The decomposition given to the user is not verifiable	Score linear in the criteria, so the decomposition of the difference is exact and can be recomputed by hand
Institutional rules understood too rigidly	Calendar rules held as configuration, promoted to hard constraints only on written confirmation by the institution
A recommendation cannot be expressed as a solver input	Recommendations restricted to a closed catalogue : weight adjustment, locked session, excluded slot or room
The language model states something false	It never produces a figure of its own; every number is checked against the context, and no decision of the platform depends on its text
The language service is unavailable or its cost rises	The service is reached through one interface which can be switched off, and every figure remains readable in its computed form

13. Appendices

- Diagram of the actors and of the main use cases.
- Diagram of the sequence from the data to the published timetable.
- Diagram of the AI recommendation and regeneration loop.
- Diagram of the system architecture and deployment.
- Diagram of the division of the responsibilities between the layers.
- Diagram of the data strategy.
- Diagram of the project workflow.
- Entity-relationship diagram of the database.
- Catalogue of the constraints, file `constraint_catalogue.csv`.
- Generator of the instance and the thirteen files produced.
- Editable sources of the figures, in Excalidraw format.

